AWS MLA-C02 • DOMAIN 4 • PART 1

Monitoring ML and AI Model Inference and Performance

Operating, Monitoring, and Securing ML and AI Solutions

01

THEORY-ONLY LEARNING SERIES

Monitor • Optimize • Secure



Production models degrade over time

A model is only as good as the data it sees today.

Data drift

The distribution of input features shifts away from the training data.

- New customer segments
- Seasonal changes
- Upstream data changes

Concept drift

The relationship between inputs and the target changes.

- Fraud patterns evolve
- Behavior shifts
- Same inputs, new answers

Pipeline errors

Failures in the systems around the model break predictions.

- Missing or null features
- Schema changes
- Container or timeout errors



SageMaker Model Monitor detects drift

Compare live traffic against a baseline on a schedule.

1 — Capture

Enable data capture on the endpoint to Amazon S3

›

2 — Baseline

Compute statistics and constraints from training data

›

3 — Schedule

Run monitoring jobs, for example hourly

›

4 — Compare

Write violation reports when data breaks constraints

›

5 — Alert

Send metrics to CloudWatch and trigger action

Model Monitor turns drift from a surprise into an alarm, and that alarm can start the retraining pipeline from Domain 3.



Model Monitor has four monitor types

Each one watches a different kind of change.

Data quality

Checks input features against the baseline statistics.

- Missing values
- Type and range checks
- Distribution shift

Model quality

Compares predictions with ground truth labels.

- Accuracy, RMSE, F1
- Needs label data
- Shows concept drift

Bias drift

Uses SageMaker Clarify to track bias metrics over time.

- Fairness by group
- Compares to baseline
- Supports governance

Feature attribution

Uses Clarify SHAP values to track feature importance.

- Which features matter
- Ranking changes
- Explains behavior



Test new models on live traffic safely

Compare a candidate against production before full rollout.

A/B testing with variants

- Several production variants on one endpoint
- Variant weights split the traffic
- TargetVariant calls a specific variant
- Compare per-variant CloudWatch metrics

Shadow testing

- A copy of traffic goes to the shadow model
- Users only see production responses
- Compare latency, errors, and outputs
- Promote the shadow when it performs



Catch errors in inference pipelines

Metrics show that something is wrong; logs show why.

Endpoint metrics

CloudWatch tracks invocations, errors, and latency.

- Invocation4XXErrors
- Invocation5XXErrors
- ModelLatency, OverheadLatency

Container logs

CloudWatch Logs stores the output of each model container.

- Stack traces
- Input parsing failures
- Out-of-memory errors

Alarms and events

Alert on thresholds and route events to automation.

- CloudWatch alarms
- EventBridge rules
- SNS notifications



Agents need step-by-step observability

An agent can fail at any step of its plan.

Agent traces

Amazon Bedrock Agents traces show each reasoning step.

- Orchestration steps
- Knowledge base lookups
- Action group calls

AgentCore Observability

Sessions, traces, and spans in CloudWatch.

- OpenTelemetry based
- Latency per step
- Token use per session

Failure signals

Patterns that point to agent problems.

- Tool call errors
- Truncated streams
- Loops and timeouts



Monitor foundation models in Amazon Bedrock

Track usage, quality, and limits for every model call.

Invocation logging

Record requests and responses to CloudWatch Logs or S3.

- Full prompt and output
- Audit and debugging
- Turned off by default

Runtime metrics

Amazon Bedrock publishes metrics to CloudWatch.

- InvocationLatency
- Input and output tokens
- InvocationThrottles

Bedrock evaluations

Score model and RAG quality with evaluation jobs.

- Automatic metrics
- LLM-as-a-judge
- Human review



A monitoring checklist

Watch the data, the model, and everything around it.

1. Capture endpoint data and create a baseline.
2. Schedule data quality and model quality monitors.
3. Test new models with variants or shadow testing.
4. Alarm on 4XX, 5XX, and latency metrics.
5. Trace agent steps and watch for tool call errors.
6. Log Bedrock invocations and evaluate quality regularly.

